Knuth's Wisdom

2012-10-12 19:32:25

Original: `https://nicknash.me/2012/10/12/knuths-wisdom/`

0.1 Or, what's really so bad about bubble sort?

1 Prologue

In a previous post (`http://nicknash.me/2012/07/31/adversaries/`) I talked about trying to bring out the worst in sorting algorithms. Given the torrent of interest in that post, I'm going to continue with the sorting theme. So, in this post, I'm going to talk about a sorting algorithm that brings out the worst in a CPU: bubble sort. This is clearly a point of deep industrial and practical interest, so, fasten your seat belts!

It turns out that the *real* reason a bubble sort performs so badly on today's CPUs is just a little surprising. Actually, I wrote about this some time ago with some friends in a paper (`http://dl.acm.org/citation.cfm?id=1370599`), but it was a long paper, and I felt this fun little bubble sort nugget got a bit lost inside it.

2 Bubble Sort

Mundane and all as it is, let's remind ourselves what bubble sort is.

For an array of n elements, bubble sort works by performing roughly n *sweeps* over the array. A sweep just scans from left-to-right, swapping an element with its immediate right neighbour if that neighbour is smaller than it.

So in C, the sweep step might look like this:

```
void maybe_swap(int* a, int i)
{
    if(a[i + 1] < a[i])
    {
        swap(a[i + 1], a[i]);
    }
}

void bubble_sweep(int* a, int n, int sweep_number)
{
    for(int i = 0; i < n - sweep_number; ++i)
    {
        maybe_swap(a, i)
```

```
    }
}
```

In all its glory, bubble sort just calls the function above with "sweep_number" ranging from 1 to $n-1$. So it looks like this:

```
void bubble_sort(int* a, int n)
{
    for(int i = 1; i < n; ++i)
    {
        bubble_sweep(a, n, i);
    }
}
```

I'm ignoring an improvement that can be made to bubble sort here. That is, if no elements are swapped in an entire sweep, the whole algorithm can just terminate. I'm leaving that improvement out because adding it makes the point I'm trying to make in this post a bit messier without adding anything very interesting.

Without getting too side-tracked into implementations, I can't resist including one in Clojure:

```
(defn maybe-swap [v i] (let [j (+ i 1)] (if (> (v i) (v j)) (swap v i j) v)))
(defn bubble-sweep [v sweep-num]
  (reduce #(maybe-swap %1 %2) v (range (- (count v) sweep-num))))
(defn bubble-sort [v] (reduce #(bubble-sweep %1 %2) v (rest (range (count v)))))
```

With our coding done, we can run a simple experiment that has a surprising result.

3 An Experiment

A quick look at the bubble sort implementation above should convince us it takes $\Theta(n^2)$ time. This is obvious: each call to "bubble_sweep" takes $\Theta(n)$ time, and we make $\Theta(n)$ calls.

If we were feeling very curious, we might try timing individual calls to "bubble_sweep" as it's called by bubble sort, just to see that the time taken is indeed linear each time. If we do that, here's what we get:

Missing diagram.
Source: `http://nicknash.me/wp-content/uploads/2012/10/bubble_sweep_cycles.png`

Link: `http://nicknash.me/wp-content/uploads/2012/10/bubble_sweep_cycles.png`

The vertical axis here is time (shown in CPU cycles), and the horizontal axis is the "sweep_number" from our code. The input array is 50,000 32-bit integers.

I don't know about you, but until now I felt like I understood the performance of the humble-looking "bubble_sweep". Now, I'm confused. Of course we *know* it's really taking $\Theta(n)$ time per sweep. It just looks like the constant hidden in there is varying pretty strangely. How come?

4 Not the Caches

A natural first thought is that maybe the number of swaps varies per sweep, and more swaps are more expensive? Maybe things are exacerbated by a CPU cache effect?

This is jumping the gun a little. It's not a cache effect, because the entire array fits in the (L2) cache where I did the measurements. The first half of this idea is good though.

The number of swaps *does* vary per sweep, and the real expense is due to branch mispredictions.

5 Due to What?

Let's quickly remind ourselves what branch mispredictions are. CPUs don't just work on a single instruction at once. Instead, they divide the work for each instruction into say, 5 stages. Then after one instruction finishes stage 1, the CPU fetches the next instruction and starts it in stage 1 (while the first instruction is now in stage 2). This is called pipelining. In this case we'd have a 5 stage pipeline.

So if processing a single instruction took time T, and we divide it into N stages, we still take time T to process a single instruction (and actually, a little longer because of the expense of administering the pipeline). On the other hand, once the pipeline is full, a new instruction is completed every T/N units of time. This is great. There's a little extra latency for each instruction, but we've improved throughput by a factor of N.

The wrinkle of course is that the CPU needs to know the *next* instruction to keep its pipeline full. When we have conditional branches (like that "if" statement in "bubble_sweep" above), the CPU can't know the next instruction. So it has to make a guess. The hardware on the CPU to do this is called a branch predictor.

When the branch predictor correctly guesses the next instruction, the pipeline can stay full. When it doesn't it needs to abort the in-flight instructions and re-start from the correct instruction. Depending on the length of the pipeline, this can be very expensive.

6 Another Experiment

Let's take a look at another experiment to try and confirm our branch prediction theory. The top-most curve in this image, labelled "hardware predictor", is the number of branch mispredictions, measured from the performance counter on the CPU, caused by the calls to "bubble_sweep" in bubble sort.

Missing diagram.
Source: `http://nicknash.me/wp-content/uploads/2012/10/bubble_sweep_mispreds.png`

Link: `http://nicknash.me/wp-content/uploads/2012/10/bubble_sweep_mispreds.png`

It certainly looks like branch mispredictions are indeed what's at work. But why?

7 Intuitive Analysis

Looking at our experimental results again, it seems like bubble sort's branches get harder and harder to predict, before becoming easier again. So what's going on?

Thinking very intuitively, we might imagine that the swapping of array elements to the right in bubble sort is a kind of partial sorting. When the data is unordered, the branch is likely to be taken, when the data is somewhat ordered, it's 50/50, and when the data is nearly sorted it's very unlikely to be taken. So perhaps it's moving between these two extremes of predictability that confounds the branch predictor.

We can do better than this though.

8 Detailed Analysis

Let's think about the probability that a particular comparison branch is taken. So let Q_l^k be the probability that the l^{th} comparison branch of the inner-loop (i.e. "bubble_sweep") is taken on the k^{th} sweep of bubble sort.

Now, the l^{th} inner-loop comparison branch can be taken only if `a[l]` is not larger than all of `a[0..l - 1]`. For this to be the case, the $(l+1)^{th}$ branch in the previous sweep must have been taken, since otherwise the previous sweep left the maximum of `a[0..l - 1]` in `a[l]`. But the probability that the $(l+1)^{th}$ branch of the previous sweep was taken is just Q_{l+1}^{k-1}.

Given that the $(l+1)^{th}$ branch of the previous sweep was taken, that probability that `a[l]` is not larger than all of `a[0..l - 1]` is $1 - 1/(l+1)$, and so we have:

$$Q_l^k = Q_{l+1}^{k-1}\left(1 - \tfrac{1}{l+1}\right)$$

The penultimate step is just a result of the fact that the probability that the l^{th} element of a random permutation is a left-to-right maximum is $1/l$.

Filling in the bases $Q_l^1 = 1 - 1/(l+1)$ for $1 \leq l < n$ and $Q_n^1 = 0$, we see that when $l \leq n - k$ then $Q_l^k = \frac{l}{l+k}$, and $Q_l^k = 0$ otherwise.

So as our experiment showed us, and our rough intuitive analysis too, in the first sweep, $k = 1$, and the branches are highly likely to be taken, on the other hand, on the last sweep $k = n - 1$ and the branches are unlikely to be taken.

Or, said another way, this analysis tells us that initially left-to-right maxima are quite rare and branches are predictable, but the repeated shifting of them to the right peppers them throughout the array making branches unpredictable, until they are dense enough that things become predictable again.

9 Matching the Hardware

To round off our mighty analysis, we can try and reproduce the curves we saw in our experiments where we measured the total number of branch mispredictions per sweep.

If we let $S(k)$ be the average total mispredictions in sweep number k, then we know $S(k) = \sum_{l=1}^{n-k} M(Q_l^k)$. Where $M(p)$ is the probability that a branch that is taken with probability p is mispredicted.

One way of modelling M is just to imagine an ideal branch predictor: if the branches are taken (independently) with probability p, then the ideal branch predictor can be wrong with probability at most $\min(p, 1-p)$. This will of course, lower bound the branch mispredictions, because in reality branch predictors work hard to guess what p is.

Filling in for the ideal predictor gives $S(k) = \sum_{l=1}^{n-k} \frac{1}{l+k} \min(l, k)$. After a lot of messy algebra we'll find that $S(k) = (1+H_n+H_k-2H_{2k})k$ when $k \leq n/2$ and $S(k) = n-(1+H_n-H_k)k$ otherwise (here $H_n \approx \ln n$ is the n^{th} harmonic number (`http://en.wikipedia.org/wiki/Harmonic_number`)).

Plotting this gives us the bottom-most curve, labelled "perfect predictor", in the branch mispredictions plot above — lower bounding the real number of mispredictions, as we expect.

10 Shaker Sort

Bubble sort has been around a long time, and people have tried to hack it into something more efficient. One attempt at this is shaker sort. In shaker sort, there are two "bubble_sweep"s that alternate. One shifts large elements right as we did above, and the other shifts small elements left (the opposite of what we did above).

Shaker sort still takes $\Theta(n^2)$ time, but is more efficient than a naive bubble sort in practice. Shaker sort is still a dog though. A selection sort implementation easily beats it, while being much simpler (and insertion sort is better still).

But, this is where the plot thickens: the reason shaker sort is **so much** slower than selection sort is almost entirely down to branch mispredictions. If you care to measure, shaker sort causes fewer cache misses than selection sort, and executes a small number of extra instructions – but performs **much worse** due to branch mispredictions. On an unpipelined machine, this bubble sort variant might actually beat selection sort.

11 So What?

If there's anything to conclude here it's that, first and foremost, bubble sort is awful! Not only is it (in naive implementations) more instruction hungry and cache unfriendly than the other quadratic-time sorting algorithms – it manages to be fantastically confusing to branch predictors, on average at least.

The relative performance of the quadratic-time sorting algorithms is important in practice. This is the reason good quicksort implementations know to switch to insertion sort (which, as it happens causes only a linear number of branch mispredictions despite executing a quadratic number of branches).

As a result, I think it's important to understand what's going on behind the simple looking code of the quadratic-time sorting algorithms. It also shows that even simple and very similar looking pieces of code can behave quite differently.

And lastly, all this messing with bubble sort reminds us of Don Knuth's near omniscience. In Volume 3 of *The Art of Computer Programming*, he remarks:

> In short, the bubble sort seems to have nothing to recommend it, except a catchy name and the fact that it leads to some interesting theoretical problems.

Knuth isn't referring to instruction pipelines when he makes this comment, but somehow he's right about how bubble sort gets on with them too!

Discussion at HackerNews (`http://news.ycombinator.com/item?id=4646509`) Code (`https://github.com/pbig`
from our paper.